PROJECT MANAGEMENT SYSTEM

Database Schema Design

Logical relational model, entity definitions, relationships and PostgreSQL reference DDL

Version 1.0 | 26 October 2026

Status: Proposed design for team review

1. Purpose and Scope

This document defines a proposed relational database schema for the Project Management System (PMS). It translates the documented project-management requirements into tables, keys, relationships, constraints and indexes that can support project/task management, workflow and Kanban status, sprint planning, estimation, comments, attachments, notifications, dashboards, reporting and activity history.

Design status: This is a proposed logical design, not evidence that the database has been created or tested. Confirm the technology stack and business rules with the team before implementation.

1.1 Scope covered

- Users and access roles for Admin, Project Manager and Team Member.
- Projects and project membership.
- Tasks/issues, assignment, priority, estimate, workflow status, sprint and milestone association.
- Workflows, states and allowed state transitions for Kanban/status movement.
- Comments, attachment metadata, notifications and activity logs.
- Supporting keys, validation rules and indexes for common retrieval patterns.

1.2 Proposed technology alignment

The schema is written for PostgreSQL and is intended to be implemented through Django models and migrations if the team confirms Django + PostgreSQL. The previously discussed architecture described this stack as a proposal; it should not be treated as finalized until the team agrees.

2. Design Decisions and Assumptions

Decision	Design approach
Relational integrity	Use primary keys, foreign keys, unique constraints and CHECK constraints to prevent invalid records.
Authentication	The users table stores a password hash only. The actual application should use the selected framework's password hashing/authentication facilities; plaintext passwords must never be stored.
Roles	Admin is a system-level privilege. Project Manager and Team Member are project membership roles stored in project_members, allowing a person to have different roles on different projects.
Workflow state	Each task has one current workflow state. Allowed moves are stored in workflow_transitions and validated by the application before a task status update.
Optional planning links	A task may be in the backlog (no sprint_id) and may or may not be associated with a milestone.
Files	The database stores attachment metadata and a storage key/path, not the binary file contents.
Dashboards and reports	Dashboard metrics and reports are computed from project/task/sprint/activity data. A separate report table is not required for the current scope.
Supporting entities	notifications, milestones, project_members and workflow_transitions are included to support requirements for notifications, sprint/milestone management, project membership and valid workflow transitions.

3. Entity-Relationship Diagram

The diagram presents the main entities and foreign-key relationships. Field-level details and rules are listed in Section 4. Dashed lines indicate contextual or actor relationships; see the data dictionary for exact foreign keys.

Diagram legend: solid arrows show direct foreign-key relationships; dashed arrows show user/actor relationships or optional contextual links. The detailed schema below defines the exact constraints.

Entity group	Tables	Purpose
Identity and access	users, projects, project_members	Accounts, project ownership and project-specific roles.
Work and workflow	tasks, workflows, workflow_states, workflow_transitions	Task records, Kanban status, and permitted status changes.
Planning	sprints, milestones	Time-boxed sprint planning and project milestones.
Collaboration and audit	comments, attachments, notifications, activity_logs	Discussion, file metadata, alerts and project history.

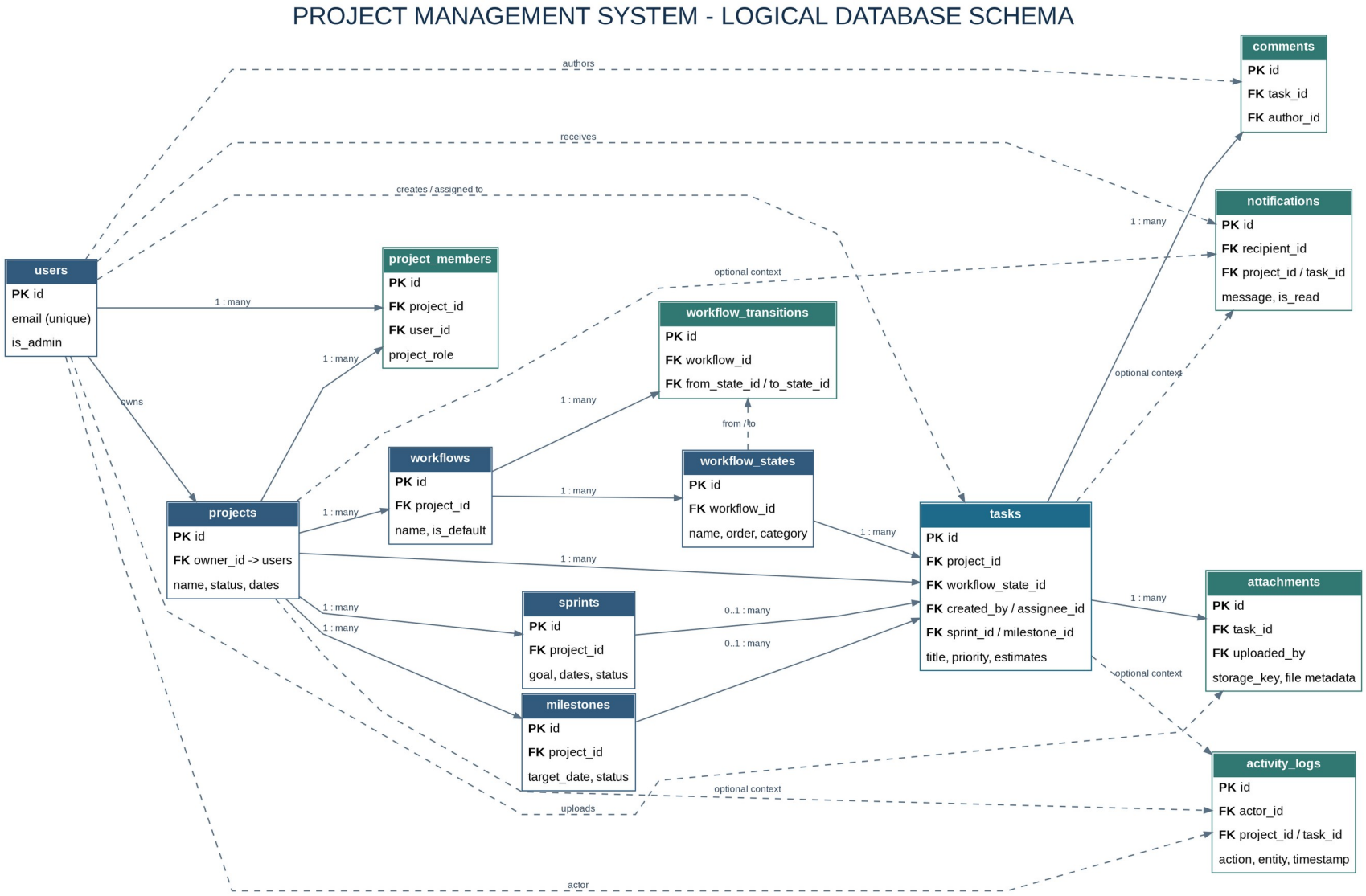


Figure 1. Proposed logical ER diagram for the Project Management System.

4. Data Dictionary

PK = primary key; FK = foreign key; UQ = unique; NN = not null. BIGINT identity columns are generated by PostgreSQL in the reference DDL. For Django, corresponding models and migrations should be the implementation source of truth.

4.1 users

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Unique user identifier.
username	VARCHAR(150)	NN, UQ	Login identifier; uniqueness enforced.
email	VARCHAR(254)	NN, UQ	Email address.
password_hash	VARCHAR(255)	NN	Encoded password hash only; never plaintext.
full_name	VARCHAR(150)	NN	Display name.
is_active	BOOLEAN	NN, default TRUE	Whether the account can access the system.
is_admin	BOOLEAN	NN, default FALSE	System-wide administrator privilege; map to framework admin flags if using Django.
created_at / updated_at	TIMESTAMPTZ	NN	Record creation and last update timestamps.

4.2 projects

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Unique project identifier.
name	VARCHAR(200)	NN	Project name.
description	TEXT	Optional	Project details.
owner_id	BIGINT	FK -> users.id, NN	Project owner; deletion restricted while projects exist.
status	VARCHAR(20)	NN, CHECK	PLANNED, ACTIVE, ON_HOLD, COMPLETED or ARCHIVED.
start_date / due_date	DATE	Optional, CHECK	Due date must not be earlier than start date if both exist.
created_at / updated_at	TIMESTAMPTZ	NN	Audit timestamps.

4.3 project_members

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Membership row identifier.
project_id	BIGINT	FK -> projects.id, NN	Project joined.
user_id	BIGINT	FK -> users.id, NN	Member user.
project_role	VARCHAR(30)	NN, CHECK	PROJECT_MANAGER or TEAM_MEMBER.
joined_at	TIMESTAMPTZ	NN	Membership start time.
(project_id, user_id)	Composite	UQ	A user appears at most once in a project membership list.

4.4 workflows

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Workflow identifier.
project_id	BIGINT	FK -> projects.id, NN	Workflow belongs to one project.
name	VARCHAR(120)	NN	Workflow name.
is_default	BOOLEAN	NN, default FALSE	Whether this is the project's default workflow.
created_by	BIGINT	FK -> users.id, nullable	User who created the workflow.
created_at / updated_at	TIMESTAMPTZ	NN	Audit timestamps.
(project_id, name)	Composite	UQ	Workflow name unique within a project.
Default workflow rule	Partial unique index	UQ	At most one default workflow per project.

4.5 workflow_states

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Workflow-state identifier.
workflow_id	BIGINT	FK -> workflows.id, NN	Workflow to which the state belongs.
name	VARCHAR(80)	NN	State name, e.g. To Do, In Progress, Done.
sequence_order	SMALLINT	NN, CHECK	Non-negative display order.
category	VARCHAR(20)	NN, CHECK	TODO, IN_PROGRESS or DONE; allows grouping on Kanban/dashboard.
is_initial / is_terminal	BOOLEAN	NN	Marks initial and terminal workflow states.
(workflow_id, name)	Composite	UQ	State name unique within a workflow.
(workflow_id, sequence_order)	Composite	UQ	No two states share an order within a workflow.

4.6 workflow_transitions

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Transition identifier.
workflow_id	BIGINT	FK -> workflows.id, NN	Workflow owning the transition.
from_state_id	BIGINT	Composite FK -> workflow_states	Source state; must belong to this workflow.
to_state_id	BIGINT	Composite FK -> workflow_states	Target state; must belong to this workflow.
created_at	TIMESTAMPTZ	NN	Creation timestamp.
(from_state_id, to_state_id)	Rule	CHECK/UQ	Self-transitions rejected; transition pair unique per workflow.

4.7 sprints

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Sprint identifier.
project_id	BIGINT	FK -> projects.id, NN	Owning project.

Field / Key	Type	Constraint	Purpose / Rule
name	VARCHAR(120)	NN	Sprint name.
goal	TEXT	Optional	Sprint goal.
start_date / end_date	DATE	Optional, CHECK	End date cannot precede start date.
status	VARCHAR(20)	NN, CHECK	PLANNED, ACTIVE or COMPLETED.
created_by	BIGINT	FK -> users.id, nullable	User who created the sprint.
created_at / updated_at	TIMESTAMPTZ	NN	Audit timestamps.
(project_id, name)	Composite	UQ	Sprint name unique within its project.

4.8 milestones

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Milestone identifier.
project_id	BIGINT	FK -> projects.id, NN	Owning project.
name	VARCHAR(160)	NN	Milestone name.
description	TEXT	Optional	Milestone notes.
target_date	DATE	Optional	Target completion date.
status	VARCHAR(20)	NN, CHECK	PLANNED, IN_PROGRESS, COMPLETED or CANCELLED.
created_at / updated_at	TIMESTAMPTZ	NN	Audit timestamps.
(project_id, name)	Composite	UQ	Milestone name unique within a project.

4.9 tasks

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Task/issue identifier.
project_id	BIGINT	FK -> projects.id, NN	Project to which the task belongs.
workflow_state_id	BIGINT	FK -> workflow_states.id, NN	Current workflow/Kanban state.
created_by	BIGINT	FK -> users.id, NN	User who created the task.
assignee_id	BIGINT	FK -> users.id, nullable	Assigned user; can be empty while unassigned.
sprint_id	BIGINT	Composite FK -> sprints, nullable	Sprint assignment; NULL means product backlog/not in a sprint.
milestone_id	BIGINT	Composite FK -> milestones, nullable	Optional milestone association.
title	VARCHAR(250)	NN	Task title.
description	TEXT	Optional	Detailed task description.
task_type	VARCHAR(20)	NN, CHECK	TASK, BUG, STORY or ISSUE.
priority	VARCHAR(20)	NN, CHECK	LOW, MEDIUM, HIGH or CRITICAL.
estimate_points	SMALLINT	Optional, CHECK	Non-negative effort/story-point estimate.
estimated_hours / actual_hours	NUMERIC(8,2)	Optional, CHECK	Non-negative hour values.
due_date	DATE	Optional	Task deadline.
created_at / updated_at / completed_at	TIMESTAMPTZ	Mixed	Creation, update and optional completion timestamps.

4.10 comments

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Comment identifier.
task_id	BIGINT	FK -> tasks.id, NN	Task being discussed.
author_id	BIGINT	FK -> users.id, NN	Comment author.
body	TEXT	NN, CHECK	Comment text cannot be blank.
created_at / updated_at	TIMESTAMPTZ	NN	Comment creation/update timestamps.

4.11 attachments

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Attachment identifier.
task_id	BIGINT	FK -> tasks.id, NN	Task associated with the attachment.
uploaded_by	BIGINT	FK -> users.id, NN	Uploader.
original_file_name	VARCHAR(255)	NN	Original filename for display.
storage_key	TEXT	NN	Storage location/key; binary content is not stored in this table.
content_type	VARCHAR(150)	Optional	MIME/content type.
size_bytes	BIGINT	NN, CHECK	Non-negative file size.
created_at	TIMESTAMPTZ	NN	Upload time.

4.12 notifications

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Notification identifier.
recipient_id	BIGINT	FK -> users.id, NN	User receiving the notification.
actor_id	BIGINT	FK -> users.id, nullable	User who triggered it, if any.
project_id	BIGINT	FK -> projects.id, nullable	Optional related project.
task_id	BIGINT	FK -> tasks.id, nullable	Optional related task.
notification_type	VARCHAR(80)	NN	Event category.
message	TEXT	NN	Notification text.
is_read	BOOLEAN	NN, default FALSE	Read/unread status.
created_at	TIMESTAMPTZ	NN	Creation time.
read_at	TIMESTAMPTZ	Optional, CHECK	Set when read; only valid when is_read is true.

4.13 activity_logs

Field / Key	Type	Constraint	Purpose / Rule
id	BIGINT	PK	Activity record identifier.
actor_id	BIGINT	FK -> users.id, nullable	User who performed the action; may be NULL if account removed.
project_id	BIGINT	FK -> projects.id, nullable	Optional project context.
task_id	BIGINT	FK -> tasks.id, nullable	Optional task context.
action	VARCHAR(100)	NN	Action label, e.g. TASK_CREATED or STATUS_CHANGED.
entity_type	VARCHAR(50)	NN	Type of object affected.
entity_id	BIGINT	Optional	ID of the affected object; polymorphic reference.
details	JSONB	NN, default empty object	Additional event data; avoid secrets.
created_at	TIMESTAMPTZ	NN	Event timestamp; activity logs should normally be append-only.

5. Relationships and Cardinalities

Relationship	Cardinality / Meaning
User -> Project	One user can own many projects; each project has one owner.
User <-> Project	Many-to-many membership implemented by project_members; each project/user pair is unique.
Project -> Workflow -> Workflow State	A project can define one or more workflows; each workflow contains multiple ordered states.
Workflow -> Workflow Transitions	Each transition connects a source state to a target state within one workflow.
Project -> Tasks	A project contains many tasks; each task belongs to one project.
Workflow State -> Tasks	Many tasks may share a state; each task has one current state.
Project -> Sprints / Milestones	A project may contain multiple sprints and milestones.
Sprint / Milestone -> Tasks	A sprint or milestone can relate to many tasks; a task can optionally be assigned to one sprint and one milestone.
Task -> Comments / Attachments	A task can have many comments and attachments.
User -> Comments / Attachments	A user can author many comments and upload many attachments.
User -> Notifications	A user can receive many notifications.
Users / Projects / Tasks -> Activity Logs	An activity record can reference an actor and optional project/task context.

6. Integrity Rules and Application Validation

- A task's workflow state must belong to a workflow associated with the same project as the task.
- A task assignee must be an active member of the task's project unless administrator policy explicitly permits otherwise.
- Project ownership and project membership must be synchronized when a project is created.
- Task status changes must follow workflow_transitions; invalid transitions must be rejected.
- A sprint or milestone linked to a task must belong to the same project as that task. Composite foreign keys in the proposed PostgreSQL DDL enforce this for sprint/milestone links.

- Only authorized users should add members, change project settings, assign tasks, modify workflows or view activity information.
- Deletion behavior should preserve auditability: task comments/attachments may cascade with task deletion, while key ownership references are restricted or set to NULL as specified.
- All timestamps should be stored consistently in UTC; the UI can render local time.

7. Indexing and Query Considerations

Index	Supports
tasks(project_id, workflow_state_id)	Project boards and status-based task lists.
tasks(assignee_id)	My Tasks and assignee filters.
tasks(sprint_id)	Sprint backlog and sprint reports.
comments(task_id, created_at)	Task discussion ordered by time.
notifications(recipient_id, is_read, created_at)	Unread notification list.
activity_logs(project_id, created_at) and activity_logs(task_id, created_at)	Project/task activity history.
project_members(user_id)	Projects available to a user.

Additional indexes should be added only after observing real query patterns. Unique constraints and foreign-key indexes should be considered alongside database size and write performance.

8. Security, Backup and Maintenance

- Use the application framework's password hashing and authentication mechanisms. Never log password values, session secrets or tokens.
- Enforce authorization in backend services, not only by hiding controls in the frontend.
- Limit attachment file types and size, validate uploads, and store files outside the public web root or in approved object/file storage.
- Use environment variables or a secret manager for database credentials; do not commit secrets to GitHub.
- Use version-controlled database migrations (for example, Django migrations) and review schema changes before merging.
- Establish periodic database backups and test restoration as part of operational readiness.
- Keep activity logs append-only to the extent practical and avoid sensitive information in JSONB details.

9. Open Decisions to Confirm Before Implementation

Decision	Needs team confirmation
Technology stack	Confirm Django + PostgreSQL. If another stack is selected, adjust types, auth tables and migration approach.
Authentication model	Confirm whether to use Django's built-in user model or a custom user model; align the users table to the selected implementation.
Role semantics	Confirm whether Project Manager/Team Member roles are project-specific. This design assumes they are.
Workflow policy	Confirm whether every project has one default workflow and whether custom workflows may be shared across projects.
Task planning	Confirm whether a task may belong to both a sprint and a milestone at the same time.
Attachment storage	Choose local development storage vs approved cloud/object storage; store only keys/metadata in PostgreSQL.
Deletion and retention	Confirm retention requirements for activity logs, removed users and completed projects.

10. Deliverables and Implementation Next Steps

1. Review this document and ER diagram with the team and record agreed decisions.
2. Translate approved entities into Django models and migrations.
3. Run migrations against a development database and verify foreign keys, unique constraints and CHECK constraints.
4. Create sample seed data for users, projects, workflows, states and tasks.
5. Add unit tests for validation constraints and integration tests for project/task/workflow relationships.
6. Commit the approved schema document, ER diagram and migration code to the shared repository.

Scope boundary: This document defines the database design. It does not claim the tables have already been created, that the SQL has been run, or that the application has passed tests.